Databricks para o projeto Ônibus SP × Chuva

Compute, ingestão, Jobs, triggers e bundles — conceito primeiro, ferramenta depois

Material de estudo · Gabriel Graciano Dias
27/09/2026

Como ler este documento

Os cinco links que o Rafael mandou cobrem cinco perguntas diferentes que todo pipeline de dados precisa responder:

Pergunta	Conceito	Nome no Databricks
Onde o código roda e quanto custa?	Compute	Serverless, clusters, SQL warehouses
Como o dado entra na plataforma?	Ingestão	Lakeflow Connect, Auto Loader
O que roda, em que ordem?	Orquestração	Lakeflow Jobs
Quando roda?	Disparo (trigger)	Schedules, file arrival, table update
Como eu reproduzo tudo isso sem clicar?	Infraestrutura como código	Declarative Automation Bundles (ex-DABs)

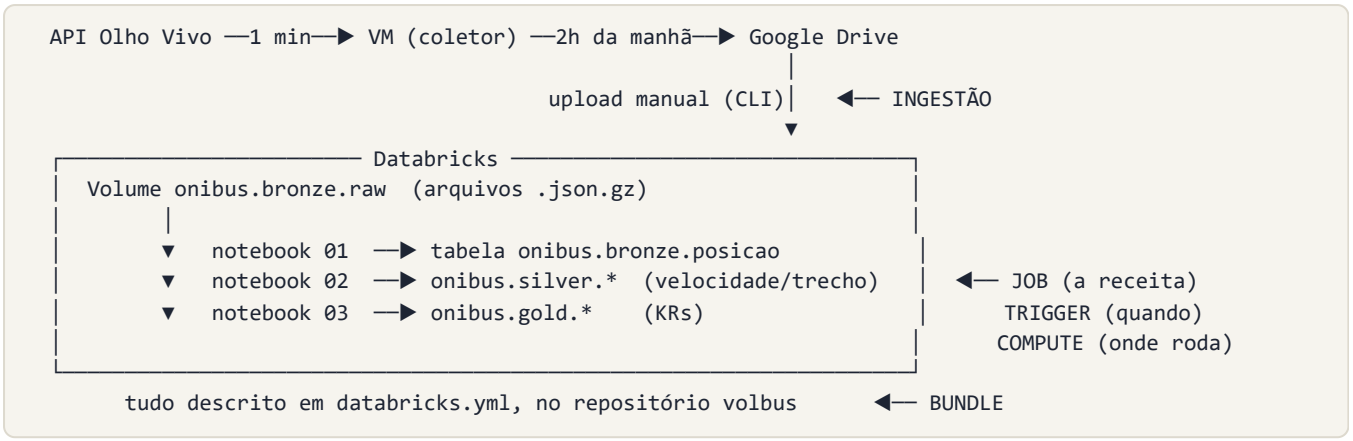
A ordem das seções segue a ordem em que você vai precisar de cada coisa no projeto: primeiro você precisa de um computador (compute), depois de dado dentro dele (ingestão), depois de uma receita que roda sozinha (job), depois de um relógio ou sensor que aperta o botão (trigger), e por fim de um jeito de guardar tudo isso em código (bundle).

Cada seção tem três partes: **o conceito** (vale para qualquer plataforma — Airflow, AWS, GCP), **como funciona no Databricks**, e **onde entra no projeto**. No fim de cada seção há perguntas para testar se você entendeu.

> Os nomes de produto do Databricks mudam com frequência (DLT virou "Lakeflow Declarative Pipelines", Workflows virou "Lakeflow Jobs", Asset Bundles virou "Declarative Automation Bundles"). O conceito é o que fica. Fatos e limites abaixo foram conferidos na documentação oficial em 27/09/2026; o que não foi confirmado está marcado com *(a confirmar)*.

O mapa do projeto

Antes dos conceitos, onde estamos — e onde cada tópico se encaixa:



Hoje só existe a parte de cima e o notebook 01, rodado à mão. O objetivo deste material é você entender como transformar isso num pipeline que roda sozinho.

1. Compute — onde o código roda

1.1 O conceito

Separar armazenamento de processamento

A ideia mais importante das plataformas de dados modernas: **o dado mora num lugar, o processamento acontece em outro, e eles só se encontram quando você precisa.**

- O **armazenamento** é barato e fica sempre ligado. No Databricks, é um "object storage" (S3 na AWS) — um disco gigante na nuvem onde ficam os arquivos das tabelas. Seus 86 milhões de posições custam centavos por mês parados ali.
- O **processamento** (compute) é caro e só deve existir enquanto está trabalhando. São máquinas virtuais que ligam, leem o dado do storage, calculam, escrevem o resultado de volta e desligam.

Consequência prática: **você paga pelo tempo de máquina ligada, não pelo tamanho do dado.** Um notebook esquecido aberto com um cluster ligado a noite toda custa mais que guardar todo o projeto por um ano.

Compare com o que você tinha antes: no seu computador, armazenamento (o HD) e processamento (a CPU e a memória) estão na mesma caixa. É por isso que a silver antiga não escalava — ela tentava trazer 86 milhões de linhas para a memória de uma máquina só.

Driver e workers: como o Spark divide o trabalho

Um cluster Spark tem dois tipos de máquina:

- **Driver** — o "gerente". Roda o seu notebook, entende o que você pediu, quebra em tarefas e distribui. Também é onde o resultado de um `.collect()` ou `.toPandas()` vai parar — por isso trazer uma tabela gigante para o driver trava tudo.
- **Workers** (executores) — os "operários". Cada um pega um pedaço do dado (uma **partição**) e processa em paralelo.

Quando você escreve `spark.read.json(...).groupBy("dt").count()`, nada acontece de imediato. O Spark monta um **plano** (avaliação preguiçosa, *lazy*). Só quando você pede um resultado (`count()`, `display()`, `write`) o driver divide o trabalho: cada worker conta as linhas das suas partições, e o driver soma os parciais.

Paralelo estatístico: é como calcular a média de uma amostra enorme dividindo em blocos, calculando soma e *n* de cada bloco em paralelo, e combinando no fim. Operações que se decompõem assim (soma, contagem, máximo) são baratas. Operações que exigem **reorganizar** o dado entre máquinas — `groupBy` por uma chave, `join`, ordenar — obrigam os workers a trocar dados pela rede. Isso se chama **shuffle** e é o que costuma deixar um job lento.

Três formas de ter compute

Forma	Analogia	Quando usar
Interativo (all-purpose)	Um carro seu, parado na garagem	Exploração em notebook, quando você fica horas testando
De job (job compute)	Um carro alugado só para a viagem	Nasce quando o job começa, morre quando termina
Serverless	Um táxi	Você não escolhe o carro; chega em segundos/minutos, paga pela corrida

Nos dois primeiros (chamados **classic**), *você* escolhe o tamanho das máquinas, quantos workers, a versão do Spark, e as máquinas rodam na sua conta de nuvem. No serverless, o Databricks mantém máquinas prontas na conta *dele* e empresta para você.

Unidade de cobrança: DBU

O Databricks cobra em **DBU** (Databricks Unit), uma unidade de "processamento por hora". Cada tipo de compute tem um preço diferente por DBU (interativo é mais caro que job; serverless embute o preço da máquina). Você não precisa decorar preços; precisa do princípio: **mesma conta rodada como job custa menos que rodada num cluster interativo**, e máquina ociosa é dinheiro jogado fora.

1.2 Como funciona no Databricks

Serverless é o padrão no seu workspace (trial com Default Storage). Existe para três usos:

- **Notebooks serverless** — o que você usou no notebook 01 ao escolher "Serverless" no Connect.
- **Jobs serverless** — o padrão quando você cria uma task de notebook sem configurar compute. Tem dois modos: *Standard* (mais barato, leva ~4–6 min para começar) e *Performance optimized* (começa rápido, consome mais DBU).
- **Pipelines serverless** — para os pipelines declarativos (seção 2).

Classic compute (clusters): você configura driver, workers, **autoscaling** (mínimo e máximo de workers — o cluster cresce quando há trabalho), **auto-termination** (desliga depois de N minutos parado) e o **access mode**: *Standard* (vários usuários, isolados) ou *Dedicated* (um usuário). **Photon** é o motor vetorizado em C++ do Databricks que acelera SQL e DataFrames; vem ligado por padrão.

SQL warehouses são compute especializado para SQL — é o que alimenta dashboards e o editor SQL. O seu workspace já tem um "Serverless Starter Warehouse" (parado). Serverless liga em poucos segundos; Pro e Classic levam ~4 min.

Limitações do serverless que vão aparecer no projeto:

- Sem Spark UI (a tela de diagnóstico de jobs lentos) e sem `cache()` / `persist()`.
- Streaming só no modo `availableNow` (processa tudo que há de novo e para) — não existe stream "sempre ligado".
- Sem RDDs, sem Scala em notebook, sem bibliotecas instaladas no cluster (instala-se por notebook/ambiente).
- Um run tem no máximo 7 dias.

- Funções Python dentro do Spark (UDFs) sem acesso à internet e com até 1 GB de memória — relevante se a projeção no shape for feita com `applyInPandas`.

1.3 No projeto

- **Use serverless para tudo agora.** É o que o trial oferece de mais simples, e o volume do projeto (~10 milhões de linhas/dia) é pequeno para Spark.
- **O custo que importa é o do M2** (projeção de posições no shape). É a única etapa pesada: para cada posição, achar o ponto mais próximo numa polilinha. A tática do plano ("1 linha e 1 dia primeiro") é também uma tática de custo.
- **Cuidado com o `toPandas()`**: trazer um dia inteiro (~10 M linhas) para o driver pode estourar a memória. Filtre (uma linha de ônibus, uma hora) antes.
- **O "problema dos arquivos pequenos"** que você viu no upload também é um problema de compute: ler 1.440 arquivos de 130 KB é mais lento que ler um de 190 MB, porque cada arquivo tem um custo fixo de abrir. A tabela Delta resolve isso — depois que o dado vira Delta, ele fica em poucos arquivos grandes em Parquet.
- **Onde o dinheiro vaza:** notebook interativo ligado sem uso. O serverless desliga sozinho depois de um tempo ocioso, mas feche a sessão quando terminar.

Para testar: (1) Por que `df.count()` é barato e `df.toPandas()` pode travar? (2) O que é um shuffle e por que um `join` entre posições e shapes provavelmente causa um? (3) Se o job de todo dia leva 10 minutos, faz diferença ele rodar em cluster interativo ou em job compute? Por quê?

2. Ingestão — como o dado entra

2.1 O conceito

Ingestão é trazer dado de uma **fonte** (API, banco, arquivos, fila de mensagens) para dentro da plataforma. Parece a parte chata, mas é onde a maioria dos pipelines quebra. Quatro ideias organizam o assunto.

Puxar (pull) ou empurrar (push)

- **Pull:** a plataforma vai até a fonte e pede. Ex.: um job que chama a API do Olho Vivo.
- **Push:** a fonte (ou alguém no meio) manda o dado para a plataforma. Ex.: a sua VM coleta e envia os arquivos.

O seu projeto é **push em duas etapas**: a VM puxa da API e empurra para o Drive; você empurra do Drive para o Databricks. Essa escolha foi deliberada (seção 2.3).

Lote (batch), incremental e streaming

- **Batch completo:** a cada execução, lê *tudo* de novo e reescreve a tabela. Simples e à prova de erro, mas o custo cresce com o histórico. É o que o notebook 01 faz hoje (`mode("overwrite")`).
- **Incremental:** a cada execução, lê **só o que chegou desde a última vez**. Exige lembrar o que já foi processado — um **estado**.
- **Streaming:** o incremental levado ao limite, com execuções tão frequentes que parecem contínuas.

A diferença entre incremental e streaming é de latência, não de natureza. No Databricks os dois usam o mesmo mecanismo (Structured Streaming); o que muda é *quando* ele roda.

Estado, checkpoint e "exatamente uma vez"

Para ser incremental, o sistema precisa saber quais arquivos já leu. Ele guarda isso num **checkpoint**: um registro persistente de "até onde fui". Se o processamento cair no meio e reiniciar, ele volta do checkpoint.

Com um checkpoint bem-feito, cada arquivo é processado **exatamente uma vez** (*exactly-once*) — nem perdido, nem duplicado. Sem ele, você tem dois riscos opostos: reprocessar tudo (duplicar linhas) ou pular o que falhou (perder linhas). A sua checagem `dif = 0` do M1 é exatamente o teste de que nenhum dos dois aconteceu.

Idempotência

Uma operação é **idempotente** se rodá-la duas vezes dá o mesmo resultado que rodá-la uma. `x = 5` é idempotente; `x = x + 5` não é.

Em pipeline, isso é vital: jobs falham e são rodados de novo. Se "rodar de novo" duplica as linhas de 22/09, a sua análise de velocidade fica errada sem nenhum erro aparecer. O notebook 01 hoje é idempotente porque reescreve tudo (`overwrite`). Quando ficar incremental, a idempotência vem do checkpoint (arquivos já lidos são pulados) ou de escrever por partição (`replaceWhere "dt = '2026-09-22'"` substitui só aquele dia).

Schema: inferir, fixar, evoluir

A fonte manda dado com uma estrutura (**schema**): quais campos, de que tipo. Três políticas:

- **Inferir:** o sistema olha uma amostra e adivinha. Prático, mas uma amostra atípica engana (um campo que só veio `null` vira texto).
- **Fixar:** você declara o schema. Foi o que o notebook 01 fez. Se a fonte mudar, você fica sabendo.
- **Evoluir:** aceitar campos novos automaticamente, ou separá-los numa coluna de "resgate" para olhar depois.

Foi por isso que o coletor v2 passou a guardar o JSON **cru**: se a SPTrans acrescentar um campo, ele não se perde no bronze, e a decisão de o que fazer com ele fica para a leitura.

2.2 Como funciona no Databricks

Lakeflow Connect é o nome guarda-chuva da ingestão. Ele organiza os conectores em níveis:

- **Managed connectors** — o Databricks cuida de tudo (autenticação, captura de mudanças, schema, retries) para fontes populares: Salesforce, Workday, HubSpot, bancos como Postgres, MySQL, SQL Server, Oracle. **Não serve para o projeto**: não existe conector para a API do Olho Vivo.
- **Standard connectors** — arquivos em object storage/volumes (via Auto Loader) e filas de mensagens (Kafka etc.). **É o que serve para o projeto.**
- **Community / custom connectors** — para uma API REST qualquer não há conector genérico pronto; o caminho oficial é escrever um conector num framework do Databricks (uma classe Python que lista tabelas, informa schema e lê dados).
- **Zerobus Ingest** — uma API de *push* em que um programa externo grava registros **direto numa tabela Delta**, sem arquivo no meio. *(A confirmar se está disponível no trial.)*

Auto Loader — o mecanismo que você vai usar

Auto Loader é o leitor incremental de arquivos. Você aponta para uma pasta (um volume), e ele processa só os arquivos novos:

```
(spark.readStream.format("cloudFiles")                # "cloudFiles" = Auto Loader
   .option("cloudFiles.format", "json")               # lê .json.gz sem configurar nada
   .option("cloudFiles.schemaLocation", CHK + "/schema")
   .load("/Volumes/onibus/bronze/raw/olhovivo/bronze_raw")
 .writeStream
   .option("checkpointLocation", CHK + "/bronze")     # o "até onde fui"
   .trigger(availableNow=True)                        # processa o que há de novo e para
   .toTable("onibus.bronze.posicao_raw"))
```

Como funciona por dentro:

1. **Descobrir arquivos novos.** Dois modos: *directory listing* (lista a pasta e compara com o que já viu — padrão, simples) ou *file notification / file events* (a nuvem avisa quando um arquivo chega — escala melhor com muitos arquivos).
2. **Lembrar o que já leu.** Guarda os nomes num banco chave-valor (RocksDB) **dentro do checkpoint**. É isso que dá o *exactly-once*.
3. **Schema.** Infere a partir dos primeiros 50 GB ou 1.000 arquivos e salva em `schemaLocation`. Em JSON, por padrão, tudo vira texto — use `cloudFiles.inferColumnTypes` ou `schemaHints`, ou passe o schema explícito, como no notebook 01. Quando aparece um campo novo, o modo padrão (`addNewColumns`) faz o stream falhar uma vez e incorporar a coluna no reinício; o modo `rescue` nunca falha e joga o inesperado numa coluna `_rescued_data`.

No serverless, só o `availableNow` é permitido — o que é perfeito para o projeto: roda, processa o que chegou, desliga.

`COPY INTO` é a versão SQL e mais simples: `COPY INTO tabela FROM '/Volumes/...' FILEFORMAT = JSON`. Também pula arquivos já carregados. Bom para cargas ocasionais; Auto Loader é o recomendado para fluxo contínuo.

Pipelines declarativos (Lakeflow Declarative Pipelines, ex-DLT): em vez de escrever *como* processar, você declara *o que* cada tabela é, e o Databricks cuida da ordem, do checkpoint e da qualidade:

```
CREATE OR REFRESH STREAMING TABLE bronze_posicao
AS SELECT * FROM STREAM read_files('/Volumes/onibus/bronze/raw/...', format => 'json');
```

Conceitos-chave: **streaming table** (cada registro processado uma vez, para fontes que só crescem) e **materialized view** (resultado recalculado quando a fonte muda). Permitem **expectations** — regras de qualidade ("velocidade entre 2 e 80 km/h") que descartam ou marcam linhas ruins e contam quantas. É uma alternativa aos notebooks + job; vale conhecer, mas não é necessário agora.

2.3 No projeto

- **Por que não coletar a API de dentro do Databricks** (recapitulando a decisão): não há conector pronto; um job a cada minuto seriam 1.440 execuções por dia, cada uma pagando o tempo de ligar; e quando o trial acabar a coleta para — dado não coletado não volta. A VM de US$ 4/mês faz isso bem. **A regra: o Databricks processa, não coleta.**
- **Hoje:** o notebook 01 é um **batch completo** sobre um dia. Serve para aprender e validar (`dif = 0`).
- **Próximo passo natural:** trocar o `spark.read` por Auto Loader com `availableNow`. Aí, a cada upload de dias novos para o volume, o job processa só os arquivos novos — e o `dif = 0` continua valendo como teste.
- **Os arquivos** `null`: com schema explícito, viram uma linha toda vazia e são filtrados. Com Auto Loader inferindo schema, poderiam confundir a inferência — mais um motivo para manter o schema declarado.
- **Os dias v1 (formato antigo)** entram por um caminho próprio (outra pasta, outro schema) e se juntam na mesma tabela — um exemplo real de *schema diferente para a mesma entidade*.
- **Futuro opcional:** com um S3 próprio (backlog), a VM poderia empurrar os arquivos direto para o bucket, tirando o Drive do caminho. Com Zerobus, empurraria registros direto para a tabela.

Para testar: (1) O notebook 01 com `overwrite` é idempotente? E se eu trocasse por `append` e rodasse duas vezes? (2) Onde o Auto Loader guarda a informação de quais arquivos já leu? O que acontece se eu apagar essa pasta? (3) Por que a coleta a cada minuto não deveria ser um job do Databricks?

3. Lakeflow Jobs — a receita que roda sozinha

3.1 O conceito

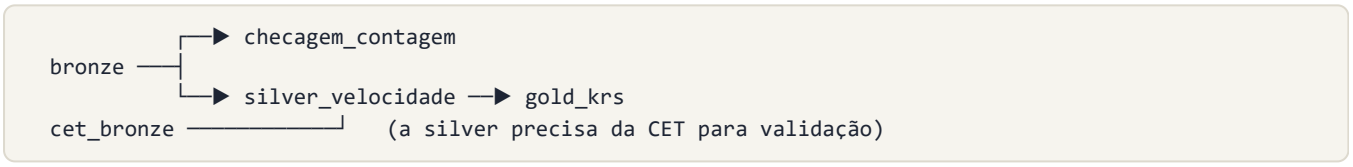
Orquestração

Um pipeline tem várias etapas com dependências: a silver só pode rodar depois da bronze; a checagem de contagem, depois da bronze; a gold, depois da silver. **Orquestrar** é garantir que as etapas rodem na ordem certa, que falhas sejam detectadas e tratadas, e que você saiba o que aconteceu.

Sem orquestrador, você é o orquestrador: abre o notebook 01, roda, espera, abre o 02, roda... e esquece um dia.

DAG: o grafo de dependências

As etapas e suas dependências formam um **DAG** (*directed acyclic graph*, grafo dirigido acíclico): setas dizem "isto depende daquilo", e não pode haver ciclo (A depende de B que depende de A). Etapas sem dependência entre si rodam em paralelo.



O que um orquestrador bom oferece

- **Retries:** tentar de novo quando uma etapa falha (útil para falhas passageiras; perigoso se a etapa não for idempotente).
- **Timeouts:** matar uma etapa que travou.
- **Parâmetros:** a mesma receita para dias diferentes (`dt = 2026-09-22`).
- **Condições:** "rode isto só se aquilo falhou" (ex.: mandar alerta).
- **Histórico:** o que rodou, quando, quanto tempo, com que resultado.
- **Reparo:** rodar de novo só o que falhou, sem refazer o que deu certo.

3.2 Como funciona no Databricks

Um **job** é um conjunto de **tasks** ligadas por dependências. Cada task é uma unidade de trabalho de um tipo:

- **Notebook** (o que você vai usar), script Python, SQL, pacote Python (wheel), JAR.
- **Pipeline** (dispara um pipeline declarativo), **dbt**, **Dashboard** (atualiza um dashboard).
- **Controle de fluxo:** *Run Job* (chama outro job), *If/else*, *For each* (repete uma task para cada item de uma lista — ex.: para cada dia).

Dependências e "run if". Cada task declara de quais depende (`depends_on`). Por padrão só roda se todas as anteriores tiveram sucesso; dá para mudar para "rode se pelo menos uma falhou" (útil para uma task de alerta) ou "rode sempre" (útil para limpeza).

Parâmetros. O job recebe parâmetros que descem para as tasks. No notebook, você lê com `dbutils.widgets.get("dt")`. Há **referências dinâmicas** que o Databricks preenche na hora:

`{{job.start_time.iso_date}}` (a data do run), `{{job.trigger.file_arrival.location}}` (qual pasta disparou).

Task values. Uma task pode passar um valor pequeno para as seguintes: `dbutils.jobs.taskValues.set(key="linhas", value=n)` na bronze, `.get(taskKey="bronze", key="linhas")` na checagem. Limite: 48 KiB, precisa ser serializável em JSON.

Falhas:

- **Retries** são desligados por padrão; você define quantas tentativas e o intervalo. Com timeout + retries, o timeout vale por tentativa. No serverless há uma otimização que já refaz tasks automaticamente — desligue se a task não for idempotente.
- **Duration thresholds:** um limite de "aviso" e um de "timeout".
- **Repair run:** refaz só as tasks que falharam e as que dependem delas. Atenção: refaz **do zero** — se a task escreveu metade dos dados antes de falhar e não é idempotente, você duplica.
- **Notificações** por e-mail (e Slack, webhook…) no início, sucesso, falha ou demora.

Concorrência. Por padrão, só **1 run por vez** de cada job; se outro disparo chegar, entra numa **fila** (até 48 h). Isso evita dois runs escrevendo na mesma tabela ao mesmo tempo.

Histórico. A tela do job mostra cada run, cada task, duração e saída do notebook. Fica guardado por **60 dias**; para mais que isso, as tabelas de sistema (`system.lakeflow.*`) guardam o histórico e cruzam com custo.

Compute do job. Sem configurar nada, tasks de notebook rodam em **serverless**. Alternativa classic: um *job cluster* compartilhado entre tasks (sobe uma vez, economiza tempo de partida).

3.3 No projeto

O primeiro job que faz sentido — e um ótimo exercício para a reunião com o Rafael:

Task	Notebook	Depende de	Observação
`bronze_posicao`	01_bronze_posicao	—	já existe
`checagem_contagem`	parte final do 01, separada	bronze_posicao	falha o job se `dif ≠ 0`
`silver_velocidade`	02 (M2)	bronze_posicao	a mais cara
`gold_krs`	03 (M3)	silver_velocidade	

Três ajustes no código para ele funcionar bem num job:

1. **Parâmetro `dt`.** Hoje o notebook lê a pasta toda. Com `dt` como parâmetro, o job processa um dia, e um *For each* processa uma lista de dias. Isso casa com o particionamento por `dt` que o coletor já faz.
2. **Idempotência por partição.** Trocar `mode("overwrite")` (que apaga a tabela inteira) por `.option("replaceWhere", f"dt = '{dt}'")`: reescreve **só aquele dia**. Rodar duas vezes o mesmo dia dá o mesmo resultado; rodar um dia não apaga os outros.
3. **A checagem precisa falhar de verdade.** Hoje ela mostra uma tabela. Num job, ela deve levantar um erro (`assert dif == 0`) — assim o job fica vermelho e você recebe o e-mail. Uma checagem que só imprime não protege ninguém.

Para testar: (1) Desenhe o DAG com a task da CET e a do KMZ. Quais podem rodar em paralelo? (2) Se a silver falhar e eu usar *repair run*, a bronze roda de novo? (3) Por que o *max concurrent runs = 1* é uma proteção útil aqui?

4. Triggers — quando o job roda

4.1 O conceito

Um job precisa de algo que aperte o botão. Duas famílias:

- **Por tempo** (*schedule*): "todo dia às 8h", "a cada 15 minutos". Previsível e simples. O risco: rodar quando não há dado novo (desperdício) ou antes do dado chegar (resultado incompleto).
- **Por evento**: "quando chegar um arquivo novo", "quando esta tabela for atualizada". Roda só quando há motivo. O risco: rodar demais, se os eventos forem muito frequentes.

A escolha é uma troca entre **latência** (quão rápido o resultado fica pronto depois que o dado chega) e **custo/complexidade**. A pergunta certa é: *de quão fresco eu preciso do resultado?*

Cron

A linguagem clássica de agendamento é o **cron**, que você já usa na VM (`*/10 * * * *` = a cada 10 min). Cada campo é uma unidade de tempo; `*` é "qualquer um", `*/10` é "a cada 10". Lembre da armadilha que você encontrou: no crontab do Linux, `%` precisava de escape.

Fuso horário

Agendamentos têm fuso. "8h" em UTC são 5h em São Paulo. E fusos com horário de verão fazem runs pularem ou repetirem na troca. É o mesmo cuidado que o projeto já tem com `ta` em UTC e partição em horário de SP.

4.2 Como funciona no Databricks

Scheduled. Modo simples (intervalo + unidade) ou avançado com cron no formato **Quartz**, que tem um campo de **segundos** a mais e usa `?` num dos campos de dia:

```
seg  min  hora  dia-mês  mês  dia-semana
 0    0    9      *       *       ?        → todo dia às 9h00
 0   0/15  *      *       *       ?        → a cada 15 min
```

Sempre informe o fuso (`America/Sao_Paulo` ou `UTC`). O agendador não é de baixa latência: runs podem atrasar alguns minutos.

File arrival. Dispara o job quando chegam arquivos novos numa pasta de um **volume** ou external location. Detalhes que importam:

- Verifica a pasta a cada ~1 minuto (melhor esforço). O custo é só o de listar a pasta.
- Só arquivos *novos* disparam; sobrescrever um existente não.
- **Sem "file events" habilitado, a pasta monitorada pode ter no máximo 10.000 arquivos.** Com file events, sem limite.
- Opções anti-rajada: *minimum time between triggers* (no máximo um disparo a cada X minutos) e *wait after last change* (espera os arquivos pararem de chegar — cada arquivo novo zera o cronômetro). Essa segunda é exatamente o que você quer para um upload de 1.440 arquivos.
- O parâmetro `{{job.trigger.file_arrival.location}}` diz ao notebook qual pasta disparou.

- A documentação recomenda combinar com Auto Loader: o trigger diz *quando*, o Auto Loader diz *quais arquivos*.

Table update. Dispara quando uma ou mais tabelas (até 10) são atualizadas — "quando a bronze mudar, rode a silver". Permite encadear jobs diferentes sem acoplar num só.

Continuous. Mantém sempre um run ativo: quando termina, começa outro. Para streaming de baixa latência. No serverless, só com `availableNow`. Não faz sentido para o projeto.

Manual. Sem trigger: roda quando você clica ou chama pela CLI/API.

4.3 No projeto

A pergunta-chave: **de quão fresco precisa ser o resultado?** A análise é retrospectiva (efeito da chuva em setembro/outubro). Ninguém precisa da velocidade de hoje às 15h03 até 15h05. Além disso, **o dado só chega ao Drive uma vez por dia** (backup das 2h, ~1h10 de duração) e ao Databricks quando você faz upload.

Portanto:

- **Agora:** trigger **manual** ou **schedule diário** (ex.: `0 0 9 * * ?`, `America/Sao_Paulo`) depois do upload. Simples e barato.
- **Por que não file arrival agora:** a 1 arquivo por minuto, a pasta passa de 10.000 arquivos em ~7 dias — o limite sem file events estoura. Contornos: arquivar os já processados, ou monitorar só a pasta do dia, ou ter file events (que um S3 próprio facilitaria). E cada disparo em serverless Standard leva minutos para começar — disparar a cada minuto seria caro e inútil.
- **Quando file arrival faria sentido:** se a VM passasse a enviar direto para um volume/S3 *um arquivo consolidado por dia* (ex.: um `.tar` ou Parquet diário), com *wait after last change*. Aí o job rodaria sozinho toda manhã, só quando o dia chegasse.
- **Table update** é útil mais adiante: job da silver dispara quando a bronze muda, job da gold quando a silver muda.
- **Atenção ao modo dev dos bundles** (seção 5): nele, todos os triggers ficam pausados de propósito.

Para testar: (1) Escreva o Quartz de "de hora em hora, no minuto 30". (2) Por que o limite de 10.000 arquivos importa tanto para o projeto e quase nada para quem recebe um arquivo por dia? (3) Que opção do file arrival evita que o job rode 1.440 vezes durante um upload?

5. Bundles — tudo isso como código

5.1 O conceito

O problema

Até aqui, você criou catálogo, schema, volume, notebook — e vai criar um job — clicando na interface ou digitando comandos soltos. Três problemas:

1. **Não é reproduzível.** Se o workspace sumir (o trial acabar), você lembra de tudo que clicou?
2. **Não é revisável.** Ninguém consegue ver, num *diff*, que o schedule mudou de 9h para 10h, nem quem mudou.
3. **Não separa ambientes.** Testar uma mudança no job "de verdade" arrisca estragar a tabela que o dashboard usa.

Infraestrutura como código

A resposta é descrever os recursos (jobs, pipelines, permissões) em **arquivos de texto versionados junto com o código**, e ter uma ferramenta que lê esses arquivos e cria/atualiza os recursos. É o mesmo princípio do `crontab.txt` que entrou no repositório hoje: em vez de o crontab existir só dentro da VM, ele está no Git, e `crontab crontab.txt` reinstala exatamente aquilo.

Ambientes

Um mesmo projeto costuma ter pelo menos dois **ambientes**:

- **dev**: onde você testa. Pode quebrar. Recursos com seu nome, agendamentos desligados.
- **prod**: o que "vale". Só muda por um processo controlado (idealmente: merge na branch principal → deploy automático).

O mesmo arquivo de configuração descreve os dois, com pequenas diferenças (qual catálogo usar, se o agendamento está ligado).

CI/CD

Continuous integration / continuous delivery: toda mudança no repositório passa por verificações automáticas e, se aprovada, é implantada sozinha. Com bundles, é um passo do GitHub Actions que roda `databricks bundle deploy -t prod` quando você dá merge na `main`.

5.2 Como funciona no Databricks

O nome atual é **Declarative Automation Bundles** (antes, *Databricks Asset Bundles*, "DABs" — o nome que o Rafael usou). Um bundle é uma pasta com um arquivo **`databricks.yml`** na raiz, que descreve o projeto:

```yaml
bundle:
  name: onibus

resources:
  jobs:
    pipeline_onibus:
      name: pipeline_onibus
      schedule:
        quartz_cron_expression: "0 0 9 * * ?"
        timezone_id: America/Sao_Paulo
      parameters:
        - name: dt
          default: "{{job.start_time.iso_date}}"
      tasks:
        - task_key: bronze_posicao
          notebook_task:
            notebook_path: ./notebooks/01_bronze_posicao.py
        - task_key: silver_velocidade
          depends_on:
            - task_key: bronze_posicao
          notebook_task:
            notebook_path: ./notebooks/02_silver_velocidade.py

targets:
  dev:
    mode: development
    default: true
  prod:
    mode: production
```

(Exemplo ilustrativo montado a partir da documentação e da API de Jobs; os notebooks 02 e o parâmetro `dt` ainda não existem.)

As partes:

- **`bundle`** — o nome do projeto.
- **`resources`** — o que será criado: jobs, pipelines, dashboards, etc. Os campos são os mesmos da API de Jobs (tudo que você configuraria na tela).
- **`targets`** — os ambientes. Um deles é o padrão.
- Também existem `variables` (ex.: `${var.catalogo}`), `permissions`, `run_as` e `include` (para dividir em vários arquivos).

Os modos fazem muito por você:

- `mode: development` — prefixa os recursos com `[dev seu_nome]`, **pausa todos os schedules e triggers**, permite runs em paralelo. Você pode experimentar sem medo de disparar nada.
- `mode: production` — confere se você está na branch Git certa, exige que se defina *em nome de quem* o job roda, e não deixa trocar o cluster na hora.

O ciclo de trabalho pela CLI (que você já tem instalada):

```bash
databricks bundle init            # cria o esqueleto a partir de um template
databricks bundle validate        # confere o databricks.yml
databricks bundle deploy -t dev   # cria/atualiza os recursos no workspace
databricks bundle run -t dev pipeline_onibus   # roda o job
databricks bundle destroy -t dev  # apaga o que o bundle criou
```

Também dá para gerar um bundle a partir de um job criado na interface (`databricks bundle generate`) — um bom caminho para aprender: crie o job clicando, depois gere o YAML e compare.

Bundles também funcionam dentro do workspace, a partir de uma **Git folder** (a pasta do workspace ligada ao GitHub), com botões de deploy e run — sem CLI. O fluxo recomendado para produção, porém, é Git + CI/CD.

5.3 No projeto

- **Ordem certa: bundle por último.** Primeiro o pipeline precisa existir e funcionar (notebooks 01–03 e um job criado à mão). Bundle é para *congelar* algo que funciona, não para descobrir o que fazer.
- **O repositório `volbus` já é o lugar.** O bundle seria o `databricks.yml` na raiz, com os notebooks em `notebooks/`. O mesmo repositório passa a ter o coletor (VM), o crontab e o pipeline (Databricks) — o projeto inteiro reproduzível a partir do Git.
- **Isso resolve o risco do trial.** Se o workspace acabar, `databricks bundle deploy` num workspace novo recria jobs e estrutura. (Os *dados* das tabelas não vêm junto — por isso o dado cru no Drive e, talvez, o S3 próprio do backlog.)
- **Dev e prod no projeto:** `dev` escrevendo num schema de teste, `prod` nas tabelas `onibus.*` que alimentam o dashboard. Para um projeto de uma pessoa, dá para começar só com `dev`.
- **Passo intermediário recomendado:** ligar o GitHub ao workspace (Git folder) antes do bundle. Os notebooks passam a vir do repositório em vez de upload manual.

Para testar: (1) Qual a diferença entre o que o `databricks.yml` guarda e o que o Drive guarda? (2) Por que o modo development pausa os schedules? (3) Se eu mudar o horário do schedule no `databricks.yml` e não rodar `deploy`, o job muda?

6. Juntando tudo: o caminho do projeto

Da situação de hoje até um pipeline completo, em passos pequenos — cada um usa um dos conceitos acima:

#	Passo	Conceito	Quando
1	Subir todos os dias (v1 e v2) e completar a bronze com `dif = 0` em todos	Ingestão (batch)	M1 — agora
2	Parametrizar o notebook 01 por `dt` e escrever com `replaceWhere`	Idempotência	M1
3	Criar à mão o job com `bronze` → `checagem` (que falha se `dif ≠ 0`)	Jobs	Reunião com o Rafael
4	Ligar o GitHub ao workspace (Git folder)	Bundles (pré-requisito)	M1/M2
5	Trocar a leitura por Auto Loader com `availableNow`	Ingestão incremental	M2
6	Acrescentar `silver` e `gold` ao job; schedule diário	Jobs + triggers	M2–M3
7	Congelar tudo num `databricks.yml` (`bundle generate` a partir do job)	Bundles	M4–M5
8	*(Opcional)* S3 próprio + envio direto da VM + file arrival	Ingestão push + triggers	Backlog

O que **não** muda em nenhum passo: a VM coleta, o dado cru fica guardado fora do Databricks, e toda etapa tem uma checagem de contagem.

7. Perguntas para levar ao Rafael

1. Quanto tempo duram os créditos do trial? (Decide o ritmo do M1 e a urgência do S3.)
2. Na prática da empresa: notebooks + Jobs, ou pipelines declarativos? Vale aprender os dois?
3. Para a projeção no shape (M2): `applyInPandas` por grupo no serverless, dado o limite de 1 GB por UDF?
4. Faz sentido criarmos o primeiro job juntos a partir do notebook 01, já com a checagem que falha?
5. Bundles: começar só com target `dev`? Ele usa GitHub Actions para deploy?

Glossário

Termo	Em uma frase
Armazenamento × compute	Dado mora barato num lugar; máquinas caras ligam só para processar
Driver / worker	O gerente que divide o trabalho / os operários que processam cada pedaço
Partição	Um pedaço do dado que um worker processa; também a divisão em pastas (`dt=`)
Shuffle	Troca de dados entre workers pela rede (em `join`, `groupBy`); costuma ser o gargalo
Lazy evaluation	O Spark só executa quando você pede um resultado
Serverless	Compute gerenciado pelo Databricks; você não escolhe máquina
DBU	Unidade de cobrança de processamento do Databricks
SQL warehouse	Compute especializado em SQL, para dashboards
Batch / incremental / streaming	Tudo de novo / só o novo / só o novo, o tempo todo
Checkpoint	Registro persistente de "até onde já processei"
Exactly-once	Cada registro processado uma vez: nem perdido, nem duplicado
Idempotente	Rodar duas vezes dá o mesmo que rodar uma
Auto Loader	Leitor incremental de arquivos do Databricks (`cloudFiles`)
`availableNow`	Modo de streaming que processa o que há de novo e para
`replaceWhere`	Escrita que substitui só as linhas que batem com um filtro (ex.: um dia)
Job / task	A receita / cada passo da receita
DAG	Grafo de dependências entre tasks, sem ciclos
Repair run	Refazer só as tasks que falharam (e as seguintes)
Trigger	O que dispara o job: horário, arquivo novo, tabela atualizada
Quartz cron	Formato de agendamento com campo de segundos
Bundle	Projeto Databricks descrito em `databricks.yml`, versionado no Git
Target	Um ambiente do bundle (dev, prod)
Git folder	Pasta do workspace ligada a um repositório Git

Fontes

Documentação oficial do Databricks (AWS), consultada em 27/09/2026:

- Bundles: https://docs.databricks.com/aws/en/dev-tools/bundles/ (e subpáginas *settings*, *deployment-modes*, *workspace*, *templates*)
- Jobs: https://docs.databricks.com/aws/en/jobs/ (e *configure-task*, *parameters*, *task-values*, *repair-job-failures*, *run-serverless-jobs*)
- Triggers: https://docs.databricks.com/aws/en/jobs/triggers (e *scheduled*, *file-arrival-triggers*, *trigger-table-update*, *continuous*)
- Ingestão: https://docs.databricks.com/aws/en/ingestion/overview (e Auto Loader, *file-detection-modes*, *schema*, COPY INTO, Zerobus, pipelines declarativos)
- Compute: https://docs.databricks.com/aws/en/compute/ (e *serverless/limitations*, *configure*, *sql-warehouse/warehouse-types*)